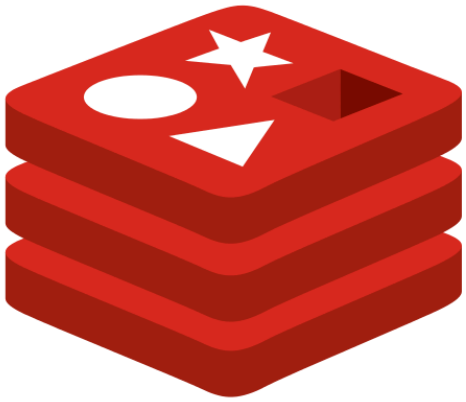
DESPLIEGUE Y SEGURIDAD DE REDIS EN CLÚSTER CON ALTA DISPONIBILIDAD



redis

Daniel Arregui Chavarría

PROYECTO FIN DE CICLO

IES Triana. 2º ASIR. Curso 2025/26

Sevilla.

RESUMEN

El presente proyecto consiste en el despliegue de un entorno de Redis distribuido, seguro y con alta disponibilidad. Redis es una base de datos in-memory, usada para caching, colas y datos transitorios, y su correcta administración es crítica en entornos empresariales. Se implementa la autenticación, cifrado de comunicaciones, replicación y se utiliza Sentinel para su supervisión y procedimiento ante failovers, además de copias de seguridad automáticas.

ÍNDICE

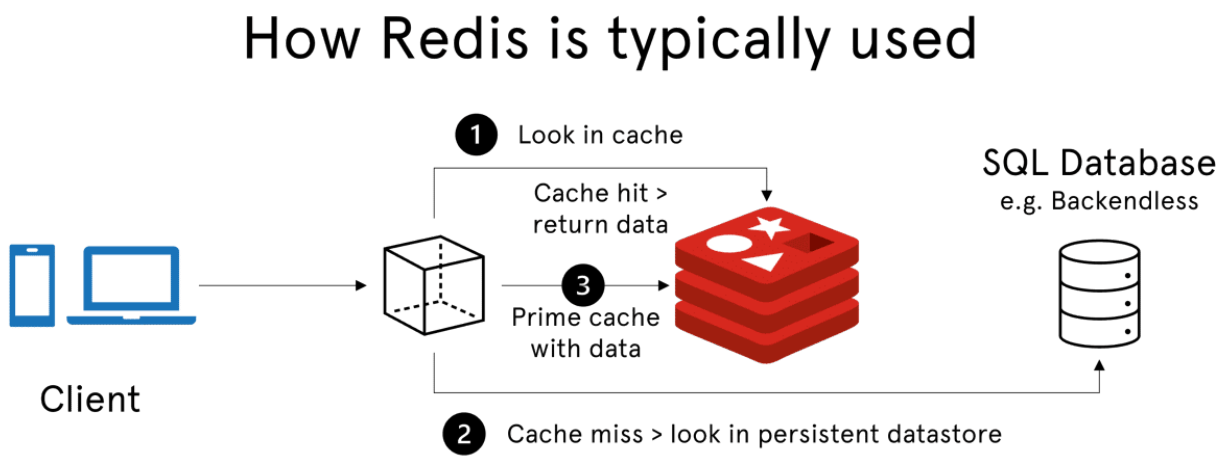
ÍNDICE

1. **INTRODUCCIÓN Y JUSTIFICACIÓN**
2. **OBJETIVOS**
3. **DESARROLLO**
4. **CONCLUSIONES**

1.- INTRODUCCIÓN Y JUSTIFICACIÓN.

¿Qué es Redis?

Redis (Remote Dictionary Server) es una base de datos en memoria, de código abierto y con un alto rendimiento, diseñada para almacenar datos estructurados en formatos simples como cadenas, listas, conjuntos, hashes y *sorted sets*. A diferencia de las bases de datos tradicionales que almacenan la información en disco, Redis mantiene los datos principalmente en memoria RAM, lo que permite tiempos de respuesta extremadamente bajos y un rendimiento muy superior para operaciones frecuentes o críticas.



Además de su rapidez, Redis puede actuar como:

- **Base de datos NoSQL clave–valor**,
- **Sistema de caché**,
- **Cola de mensajes**,
- **Contador distribuido**,
- **Almacén de sesiones**,
- **Broker de Pub/Sub**, entre otros.

Redis también ofrece opciones de persistencia (RDB y AOF), replicación maestro–esclavo, soporte para scripts Lua y múltiples estructuras de datos optimizadas.

¿Por qué configurar Redis en cluster?

La configuración en clúster, conocida como **Redis Cluster**, permite desplegar múltiples nodos Redis conectados entre sí para ofrecer una arquitectura distribuida, escalable y tolerante a fallos. Esta modalidad agrega una serie de ventajas clave respecto a una instalación individual:

1. Escalabilidad horizontal (sharding automático)

Redis Cluster divide automáticamente los datos en **16384 slots** y los distribuye entre varios nodos.

Esto permite:

- Repartir la carga de trabajo.
- Aumentar la capacidad total de memoria disponible.
- Mejorar el rendimiento general del sistema.

Cada nodo es responsable de un conjunto de slots, lo que permite añadir o eliminar nodos sin interrupciones.

2. Alta disponibilidad y tolerancia a fallos

Cada nodo principal del clúster puede tener uno o más **nodos réplica**.
Si un nodo principal falla, el clúster:

- **Promueve automáticamente una réplica** al nodo principal.
- Sigue funcionando sin detener las operaciones.

Esto reduce el riesgo de interrupciones del servicio y proporciona resiliencia ante fallos físicos o lógicos.

3. Operaciones distribuidas sin necesidad de un coordinador externo

Redis Cluster no requiere Zookeeper, Etcd ni servicios adicionales para gestionar la coordinación.
Los nodos:

- Se comunican directamente entre sí.
- Comparten información del estado del clúster mediante un protocolo interno.
- Pueden detectar fallos sin depender de otros sistemas.

4. Balanceo de carga

Al distribuir las claves entre varios nodos, el tráfico se reparte automáticamente, evitando que un único servidor se convierta en un cuello de botella. Esto es especialmente importante en aplicaciones con miles o millones de consultas por segundo.

5. Mayor capacidad total de almacenamiento

Al sumar la RAM de todos los nodos del clúster, Redis puede almacenar mucha más información que una instancia individual, lo que hace viable su uso en sistemas de gran escala.

Resumiendo:

- **Redis** es una base de datos en memoria extremadamente rápida, ideal para caché, colas, sesiones y datos clave–valor.
- **Un clúster de Redis** añade escalabilidad, alta disponibilidad, tolerancia a fallos, balanceo de carga y mayor capacidad de almacenamiento sin necesidad de herramientas externas.

2.- OBJETIVOS.

El objetivo de este proyecto consiste en desplegar un entorno de Redis distribuido, seguro y altamente disponible. Redis es una base de datos in-memory, usada para caching, colas y datos transitorios, y su correcta administración es crítica en entornos empresariales.

Objetivos generales

- Desplegar un clúster de Redis seguro, cifrado y tolerante a fallos.
- Garantizar alta disponibilidad mediante Redis Sentinel.
- Configurar políticas de control de acceso y autenticación.
- Implementar copias de seguridad automáticas para recuperación ante fallo.
- Aprender la administración de Redis en entornos empresariales.

Objetivos específicos

1. Instalar Redis en múltiples nodos (al menos tres VMs).
2. Configurar replicación maestro-esclavo entre nodos.
3. Habilitar Redis Sentinel para supervisión y failover automático.
4. Configurar autenticación mediante requirepass y masterauth.
5. Activar cifrado de comunicaciones:
    - Con stunnel para versiones anteriores.
    - Con TLS nativo si Redis soporta TLS.
6. Implementar backups automáticos usando redis-cli SAVE o BGSAVE, y almacenamiento seguro.

7. Probar el comportamiento ante caídas de nodo y restauración de backups.

8. Documentar la configuración, pruebas y resultados.

Tecnologías y herramientas

Como tecnologías y herramientas a utilizar, se contemplan las siguientes:

Herramienta / Servicio	Función
Redis	Base de datos in-memory distribuida
Redis Sentinel	Alta disponibilidad y failover automático
TLS / stunnel	Cifrado de comunicaciones entre clientes y servidores
Ubuntu Server	Sistema operativo en VMs
cron / scripts	Automatización de backups y tareas administrativas
iptables / UFW	Restricción de acceso a nodos y puertos

Arquitectura del sistema (VMs simuladas)

- VM1 – Nodo maestro Redis
- VM2 – Nodo esclavo Redis
- VM3 – Nodo esclavo Redis
- VM4 – Sentinel (puede ser distribuido en los nodos existentes)
- VM5 – Cliente de pruebas / generador de carga

Todos los nodos conectados en red interna y protegidos mediante firewall.

Fases del proyecto

La ejecución del proyecto se va a estructurar en las siguientes fases.

1. Instalación y configuración base de Redis

- Instalar Redis en cada nodo.
- Configurar redis.conf para:
  - port y bind
  - requirepass y masterauth
  - Persistencia: appendonly yes o save intervalos
  - Logs y directorios de datos

2. Configuración de replicación

- Definir un nodo maestro y dos esclavos.
- Configurar esclavos con replicaof <IP> <port> y autenticación masterauth.

Verificar replicación de datos:

redis-cli -h <slave_ip> -a <password> info replication

-

3. Alta disponibilidad con Redis Sentinel

- Configurar Sentinel en al menos 3 instancias para monitoreo.
- Parámetros importantes en sentinel.conf:
  - sentinel monitor <master_name> <ip> <port> <quorum>

    - sentinel auth-pass <master_name> <password>

- Simular fallo del nodo maestro → Sentinel promueve un esclavo automáticamente.
- Verificar reconexión de clientes al nuevo maestro.

4. Seguridad y cifrado

- Autenticación de usuarios mediante requirepass.
- Cifrado TLS nativo (o Stunnel si la versión no lo soporta).
- Limitación de acceso por IP y firewall (iptables / UFW).
- Verificación de que conexiones no autorizadas son rechazadas.

5. Backups automáticos

- Crear scripts de backup que ejecuten:
    - redis-cli SAVE o BGSAVE
    - Copia del archivo .rdb a directorio seguro
    - Opcional: cifrado con GPG
- Programar backups con cron.
- Verificación de restauración en un nodo esclavo o nodo independiente.

6. Pruebas y validación

- Inserción y lectura de datos desde cliente.
- Escritura en maestro y replicación en esclavos.
- Simulación de fallo del maestro → Sentinel promueve esclavo.

- Intento de acceso sin contraseña → denegado.
- Conexión sin TLS → denegado si se habilita cifrado.

Restauración de backup → verificación de integridad de datos.

3.- DESARROLLO.

3.1.- instalación redis en Ubuntu 22.04

1.Instalación y configuración base de Redis

1.1. Instalación Redis en cada nodo:

```
daniel@daniel-VirtualBox:~$ sudo apt install redis-server -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  libjemalloc2 liblua5.1-0 liblzf1 lua-bitop lua-cjson redis-tools
Paquetes sugeridos:
  ruby-redis
Se instalarán los siguientes paquetes NUEVOS:
  libjemalloc2 liblua5.1-0 liblzf1 lua-bitop lua-cjson redis-server
  redis-tools
0 actualizados, 7 nuevos se instalarán, 0 para eliminar y 294 no actualizados.
```

1.2. Configurar redis.conf:

1.2.1. Port y bind:

En /etc/redis/redis.conf, cambiamos:

127.0.0.1 ::0 por → 0.0.0.1 :: 1

```
daniel@daniel-VirtualBox: ~
  GNU nano 6.2                    /etc/redis/redis.conf *
# ~~~ WARNING ~~~ If the computer running Redis is directly exposed to the
# internet, binding to all the interfaces is dangerous and will expose the
# instance to everybody on the internet. So by default we uncomment the
# following bind directive, that will force Redis to listen only on the
# IPv4 loopback interface address (this means Redis will only be able to
# accept client connections from the same host that it is running on).
#
# IF YOU ARE SURE YOU WANT YOUR INSTANCE TO LISTEN TO ALL THE INTERFACES
# JUST COMMENT OUT THE FOLLOWING LINE.
# ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
bind 0.0.0.1 ::1

# Protected mode is a layer of security protection, in order to avoid that
# Redis instances left open on the internet are accessed and exploited.
#
# When protected mode is on and if:
```

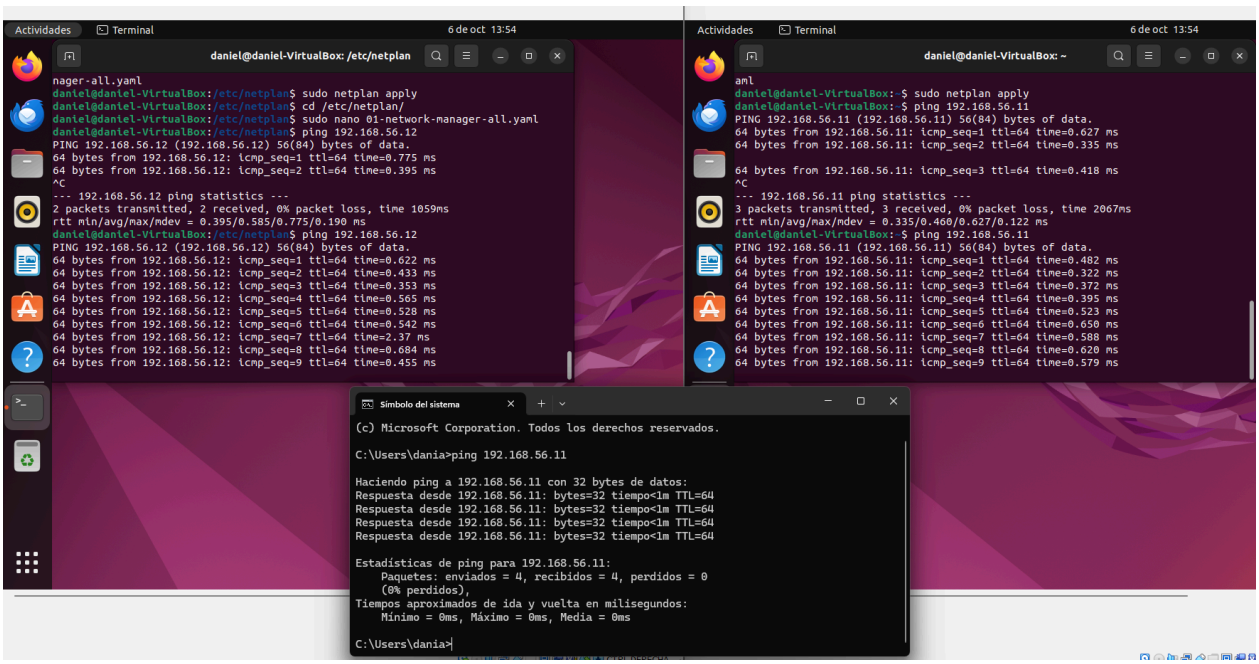
Ponemos IPs fijas (estáticas) en las dos VMs Ubuntu Desktop 22.04.

Esto es esencial para que Redis Master y Esclavo puedan comunicarse siempre con la misma dirección (por ejemplo, 192.168.56.11 y 192.168.56.12).

Primero modificamos el archivo que se encuentre en /etc/netplan y lo dejamos así (haremos lo mismo en la otra máquina pero será 192.168.56.11):

```
daniel@daniel-VirtualBox: ~
  GNU nano 6.2          /etc/netplan/01-network-manager-all.yaml
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: no
      addresses:
        - 192.168.56.12/24
```

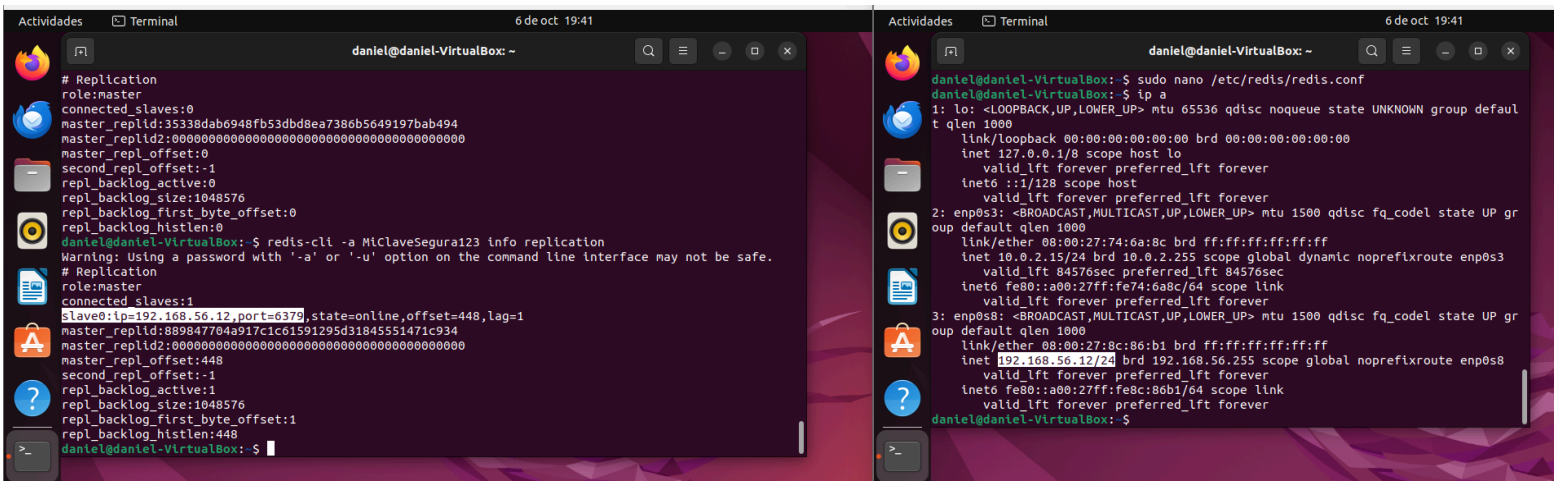
Y comprobamos la conexión de las 3 máquinas.



1.2.2. Requirepass y masterauth:

En la máquina “máster” configuramos la “requirepass” y en la máquina “esclavo” configuramos “replicaof” poniendo la ip de la máquina máster, en mi caso 192.168.56.11 y el puerto 6379 y la variable “masterauth” en la que ponemos la contraseña que anteriormente pusimos en la máster.

Luego para comprobar que se ha realizado con éxito (vemos que en la máster dice que tiene un esclavo y pone su ip: 192.168.56.12):



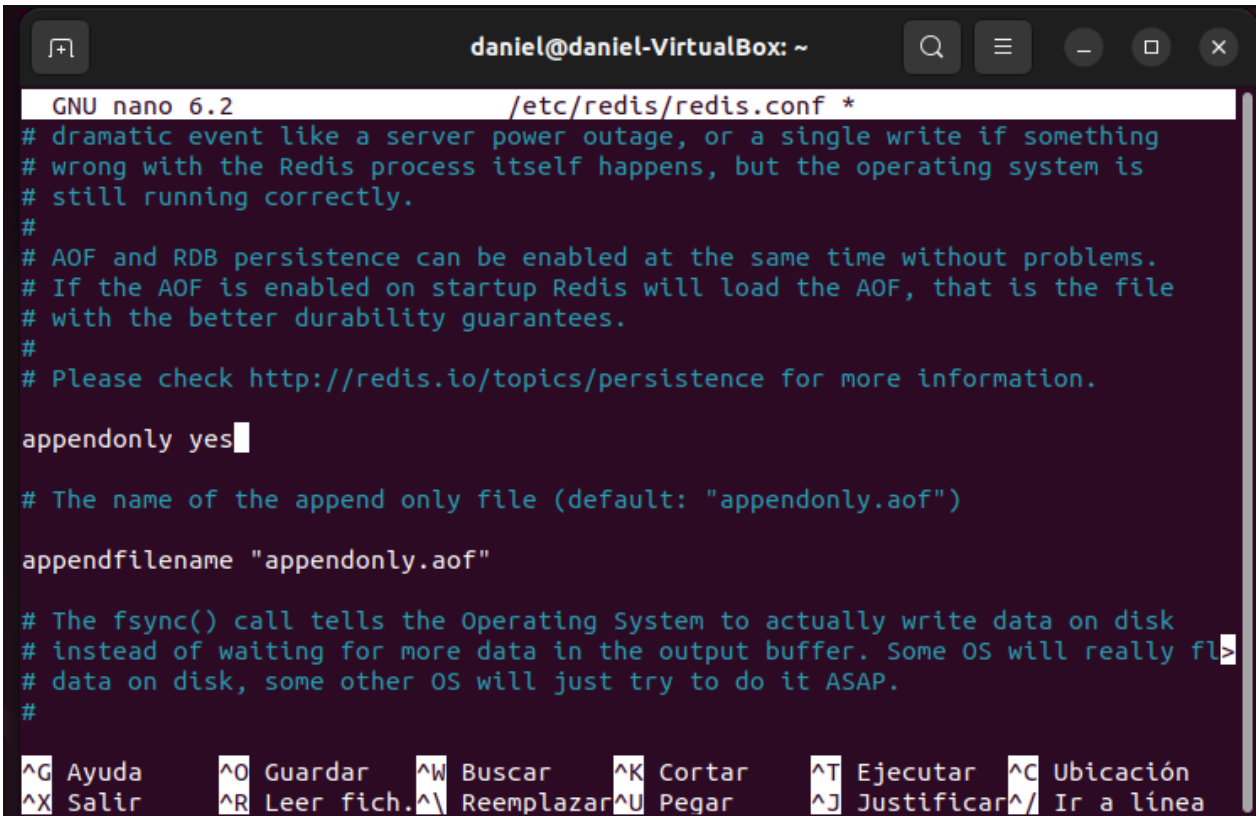
1.2.3. Persistencia: appendonly yes y/o save intervalos:

Configuramos las opciones de “appendonly” y “save” en el archivo de configuración de redis:

Appendonly:

appendonly yes → activa el registro de todas las operaciones.

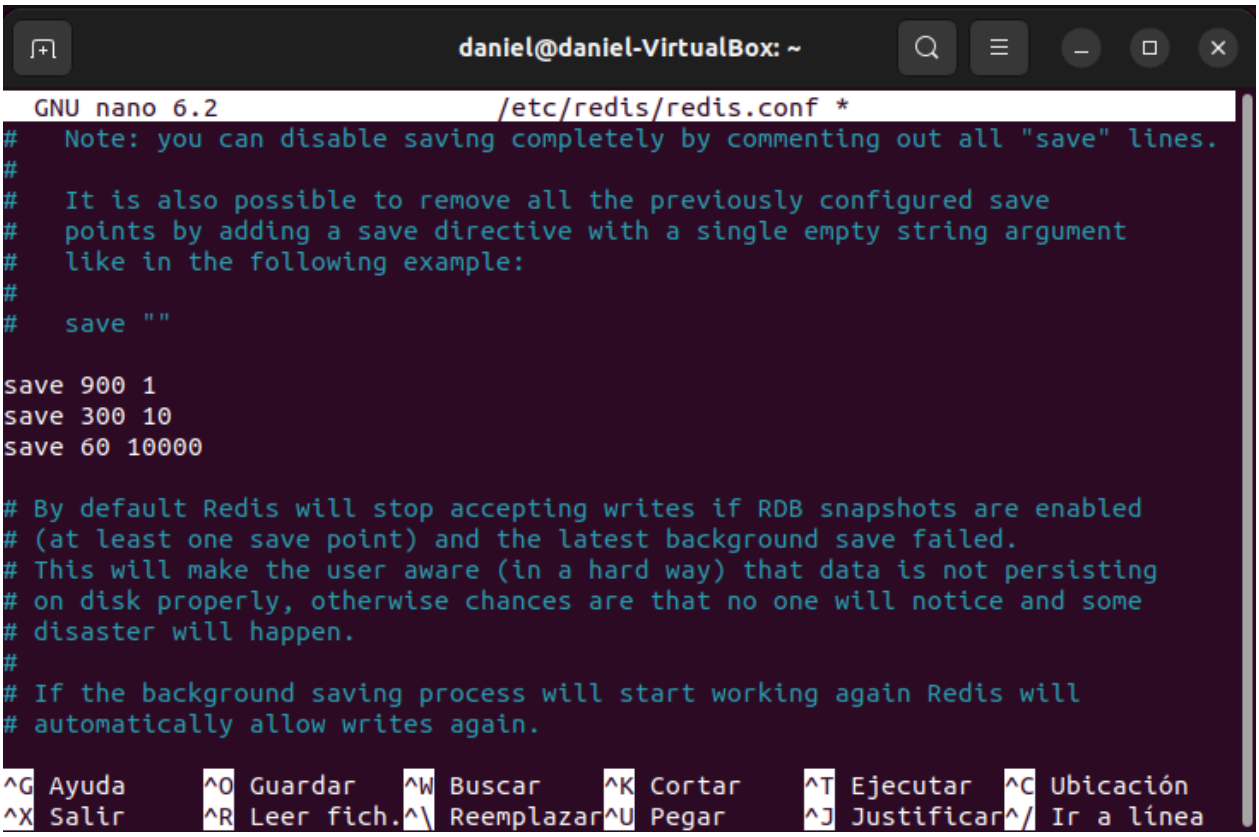
appendfsync everysec → equilibra seguridad y rendimiento.



Save:

- Guarda un snapshot RDB cada:
    - 1 cambio en 900 segundos,

- 10 cambios en 300 segundos,
- 10 000 cambios en 60 segundos.



1.2.4. Logs y directorios de datos:

En el archivo de configuración de redis buscamos y configuramos las siguientes líneas:

dir /var/lib/redis

logfile /var/log/redis/redis-server.log

Reinicia Redis

Después de hacer los cambios:

sudo systemctl restart redis-server

sudo systemctl status redis-server

Y comprueba conectividad:

redis-cli -a MiClaveSegura123 info replication

```
daniel@daniel-VirtualBox:~$ redis-cli -a MiClaveSegura123 info replication
Warning: Using a password with '-a' or '-u' option on the command line interface may n
ot be safe.
# Replication
role:master
connected_slaves:0
master_replid:77be1fb774da27048681f9ec563343c949cdd5ee
master_replid2:0000000000000000000000000000000000000000
master_repl_offset:0
second_repl_offset:-1
repl_backlog_active:0
repl_backlog_size:1048576
repl_backlog_first_byte_offset:0
repl_backlog_histlen:0
daniel@daniel-VirtualBox:~$
```

Verifica logs y persistencia

Para revisar los logs:

sudo tail -f /var/log/redis/redis-server.log

Para comprobar que los archivos de persistencia existen:

ls -lh /var/lib/redis/

Deberíamos ver:

dump.rdb

appendonly.aof

```
daniel@daniel-VirtualBox:~$ sudo tail -f /var/log/redis/redis-server.log
 |`-._`-._    `-.__.-'    _.-'_.-'|
 |    `-._`-._        _.-'_.-'    |
  `-._    `-._`-.__.-'_.-'    _.-'
      `-._    `-.__.-'    _.-'
          `-._        _.-'
              `-.__.-'

1931:M 11 Oct 2025 13:51:40.933 # Server initialized
1931:M 11 Oct 2025 13:51:40.933 # WARNING overcommit_memory is set to 0! Background sa
ve may fail under low memory condition. To fix this issue add 'vm.overcommit_memory =
1' to /etc/sysctl.conf and then reboot or run the command 'sysctl vm.overcommit_memory
=1' for this to take effect.
1931:M 11 Oct 2025 13:51:40.933 * Ready to accept connections
^C
daniel@daniel-VirtualBox:~$ ls -lh /var/lib/redis/
ls: no se puede abrir el directorio '/var/lib/redis/': Permiso denegado
daniel@daniel-VirtualBox:~$ sudo ls -lh /var/lib/redis/
total 4,0K
-rw-r----- 1 redis redis  0 oct 11 13:51 appendonly.aof
-rw-rw---- 1 redis redis 93 oct 11 13:51 dump.rdb
daniel@daniel-VirtualBox:~$
```

2.Configuración de replicación:

2.1. Definir un nodo maestro y un esclavo:

En el punto 1.2.2. Al configurar el apartado de “replicaof” el nodo donde se cambió esta configuración pasa a ser *réplica* o *esclavo*.

2.2. Configurar esclavos con “replicaof” <IP> <port> y autenticación masterauth:

En el archivo de configuración de Redis modificamos la línea donde pone “replicaof” y la dejamos de la siguiente manera:

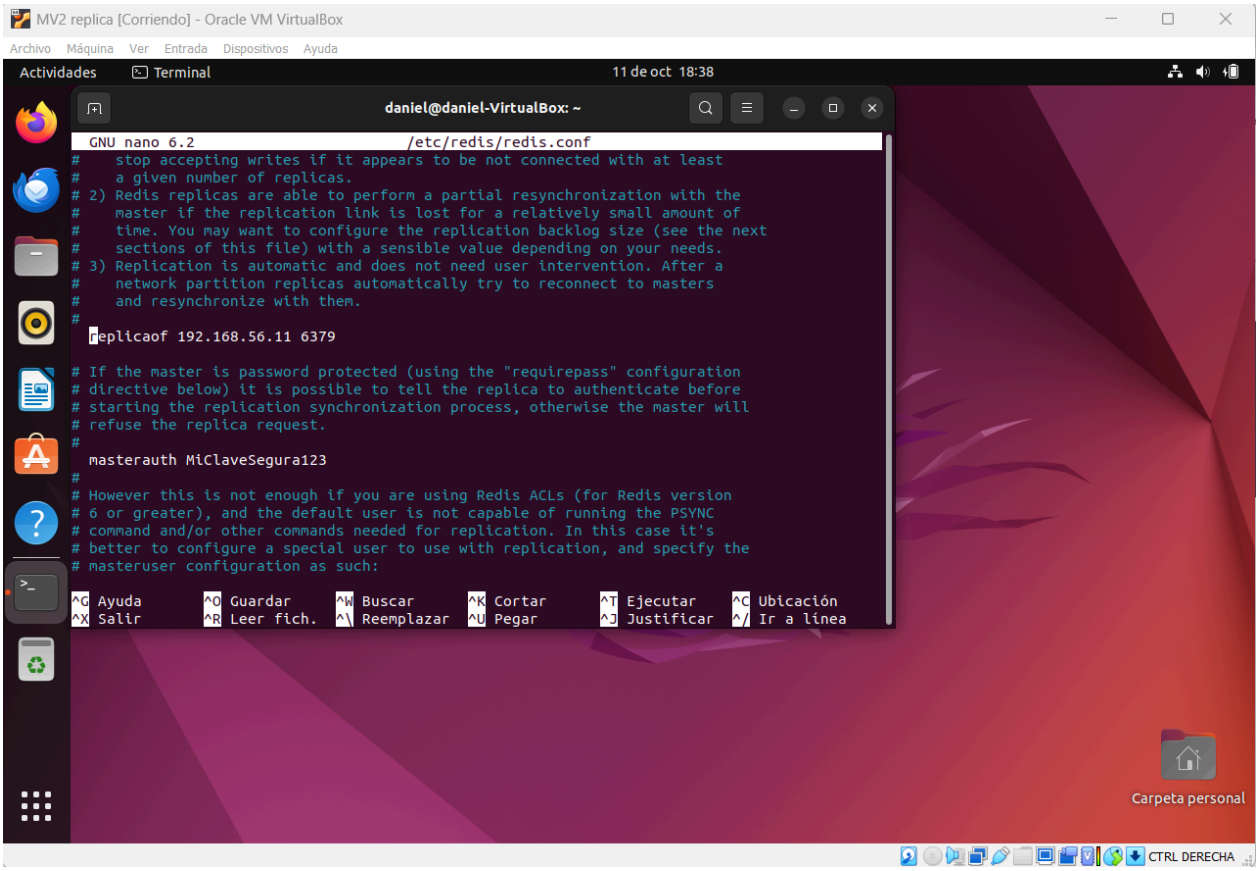
replicaof 192.168.56.11 6379

Dónde:

192.168.56.11 → IP del nodo maestro.

6379 → Puerto de Redis

Y en la línea donde tenemos masterauth sustituimos la que viene por defecto y ponemos la que hemos configurado en el nodo maestro. Quedando así la configuración del nodo esclavo:



3- Alta disponibilidad con redis sentinel

3.1. Configuración sentinel:

Para configurar Sentinel en al menos 3 instancias usaremos la máquina donde tenemos el nodo maestro, la del nodo esclavo y crearemos una tercera máquina nueva.

Comprobamos si tenemos Redis Sentinel instalado y si no lo está lo hacemos con el siguiente comando:

sudo apt install redis-sentinel -y

Creamos o modificamos el archivo de configuración de Sentinel (sudo nano /etc/redis/sentinel.conf) y debe dejarse de la siguiente manera:

```
daniel@daniel-VirtualBox: /etc
  GNU nano 6.2                    /etc/redis/sentinel.conf *
port·26379
sentinel·monitor·redis-master·192.168.56.11·6379·3
sentinel·auth-pass·redis-master·MiClaveSegura123
sentinel·down-after-miliseconds·redis-master·5000
sentinel·failover-timeout·redis-master·10000
sentinel-syncs·redis-master·1

^G Ayuda      ^O Guardar    ^W Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^J Justificar ^/ Ir a línea
```

port 26379 → **Puerto** en el que escucha Sentinel.

sentinel monitor redis-master 192.168.56.11 6379 2 → Esta es **la línea más importante**.

Le dice a Sentinel qué Redis va a vigilar como master y cuántos **Sentinels** deben estar **de acuerdo** para iniciar un **failover**. En nuestro caso 3.

sentinel auth-pass redis-master MiClaveSegura123 → Indica la **contraseña que Sentinel usará para autenticarse** con el master (y sus réplicas).

sentinel down-after-milliseconds redis-master 5000 → Define **cuánto tiempo (en milisegundos)** debe pasar sin respuesta del master para considerarlo *"down"* (caído).

sentinel failover-timeout redis-master 60000 → Controla **cuánto tiempo (en milisegundos)** dura todo el proceso de failover y las operaciones relacionadas.

sentinel parallel-syncs redis-master 1 → Controla **cuántas réplicas pueden sincronizarse con el nuevo master al mismo tiempo** durante un failover.

Puede que más adelante nuestra configuración aparezca de la siguiente manera automáticamente:

```
daniel@daniel-VirtualBox: ~
  GNU nano 6.2                    /etc/redis/sentinel.conf
port 26379
sentinel myid c2c7e9c8456b63d8999738386bc91a79544e3b3b
sentinel deny-scripts-reconfig yes

# Generated by CONFIG REWRITE
protected-mode no
user default on nopass ~* +@all
dir /var/lib/redis-sentinel
sentinel monitor redis-master 192.168.56.11 6379 3
sentinel auth-pass redis-master MiClaveSegura123
sentinel config-epoch redis-master 0
sentinel leader-epoch redis-master 0
sentinel known-replica redis-master 192.168.56.12 6379
sentinel known-sentinel redis-master 192.168.56.11 26379 4d70dedfc34a7fa8945431554df00df0a1>
sentinel current-epoch 0

                              [ 15 líneas leídas ]
^G Ayuda      ^O Guardar    ^W Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^J Justificar ^/ Ir a línea
```

Significa que Redis Sentinel ha modificado automáticamente su propia configuración en tiempo de ejecución.

Esa línea # Generated by CONFIG REWRITE te está diciendo que el archivo fue reescrito internamente por Sentinel para reflejar el estado actual del cluster.

Ejecutamos Redis Sentinel para ver que funciona con el comando:

sudo redis-sentinel /etc/redis/sentinel.conf

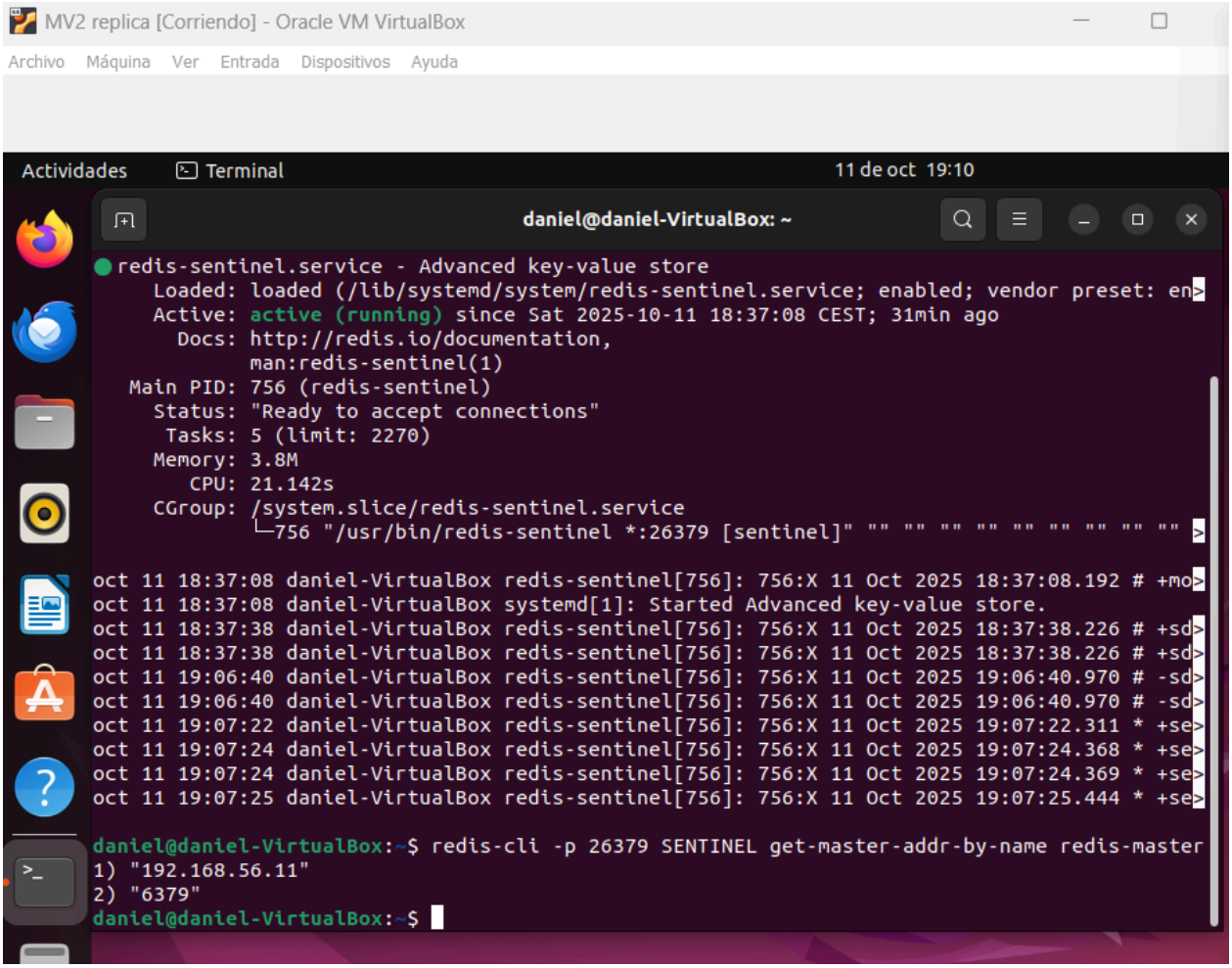
```
daniel@daniel-VirtualBox:~$ sudo redis-sentinel /etc/redis/sentinel.conf
2344:X 06 Oct 2025 20:43:44.161 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
2344:X 06 Oct 2025 20:43:44.161 # Redis version=6.0.16, bits=64, commit=00000000
, modified=0, pid=2344, just started
2344:X 06 Oct 2025 20:43:44.161 # Configuration loaded
2344:X 06 Oct 2025 20:43:44.161 * Increased maximum number of open files to 1003
2 (it was originally set to 1024).
                _.-``__ ''-._
           _.-``    `.  `_.  ''-._           Redis 6.0.16 (00000000/0) 64 bit
       .-`` .-```.  ```\/    _.,_ ''-._
      (    '      ,       .-`  | `,    )     Running in sentinel mode
      |`-._`-...-` __...-.``-._|'` _.-'|     Port: 26379
      |    `-._   `._    /     _.-'    |     PID: 2344
       `-._    `-._  `-./  _.-'    _.-'
      |`-._`-._    `-.__.-'    _.-'_.-'|
      |    `-._`-._        _.-'_.-'    |           http://redis.io
       `-._    `-._`-.__.-'_.-'    _.-'
      |`-._`-._    `-.__.-'    _.-'_.-'|
      |    `-._`-._        _.-'_.-'    |
       `-._    `-._`-.__.-'_.-'    _.-'
           `-._    `-.__.-'    _.-'
               `-._        _.-'
                   `-.__.-'

2344:X 06 Oct 2025 20:43:44.162 # Sentinel ID is 67d304a0e50478495bd0851316cd347
4b6601997
2344:X 06 Oct 2025 20:43:44.162 # +monitor master redis-master 192.168.56.11 637
9 quorum 3
```

Haremos esto mismo en las 3 máquinas (Máster, esclavo y sentinel) para tener 3 instancias de monitoreo.

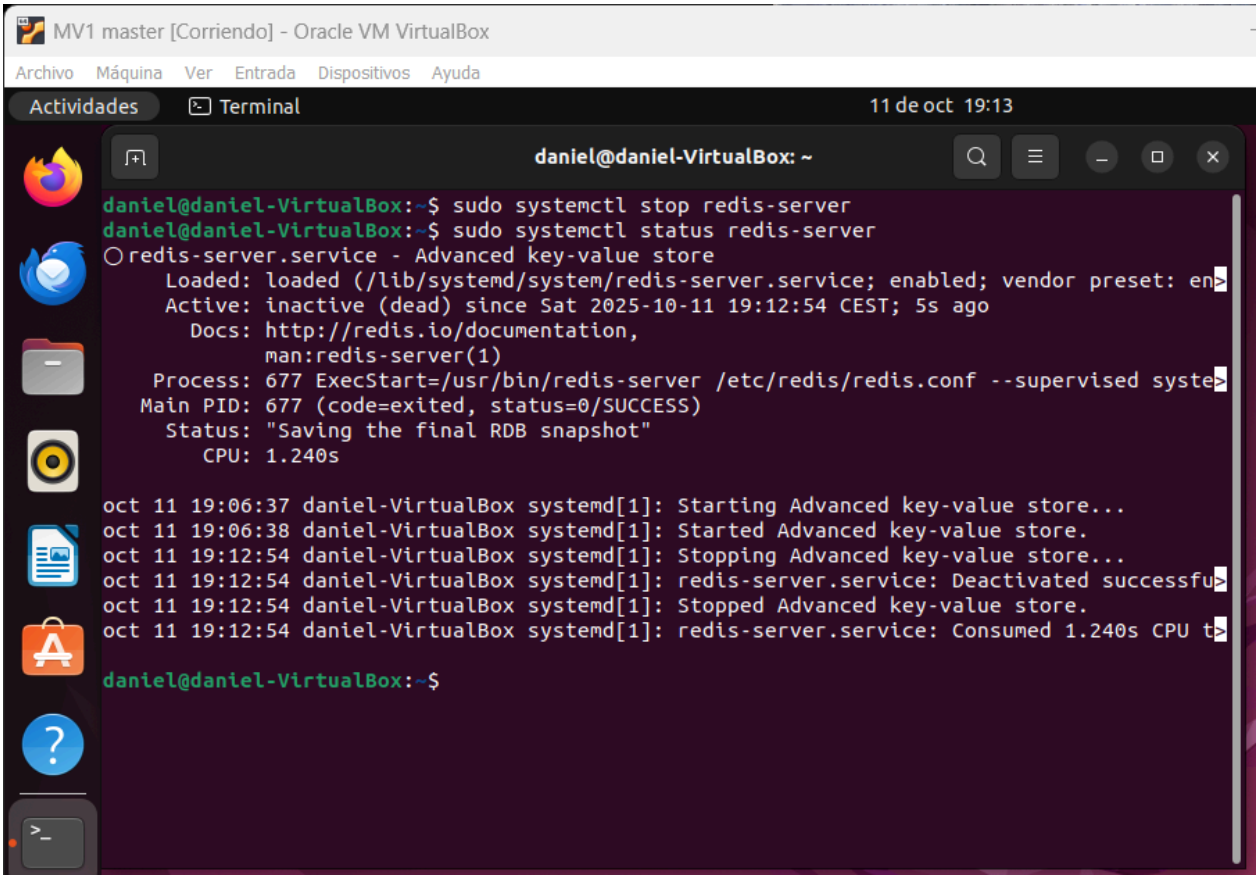
3.2. Simular fallo del nodo maestro → Sentinel promueve un esclavo automáticamente.

Confirmamos el maestro desde uno de los otros dos nodos:



Desde la máquina maestro paramos Redis con el siguiente comando:

sudo systemctl stop redis.service


MV1 master [Corriendo] - Oracle VM VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
Actividades
Terminal
11 de oct 19:13
daniel@daniel-VirtualBox: ~
daniel@daniel-VirtualBox:~$ sudo systemctl stop redis-server
daniel@daniel-VirtualBox:~$ sudo systemctl status redis-server
redis-server.service - Advanced key-value store
Loaded: loaded (/lib/systemd/system/redis-server.service; enabled; vendor preset: en
Active: inactive (dead) since Sat 2025-10-11 19:12:54 CEST; 5s ago
Docs: http://redis.io/documentation,
man:redis-server(1)
Process: 677 ExecStart=/usr/bin/redis-server /etc/redis/redis.conf --supervised syste
Main PID: 677 (code=exited, status=0/SUCCESS)
Status: "Saving the final RDB snapshot"
CPU: 1.240s
oct 11 19:06:37 daniel-VirtualBox systemd[1]: Starting Advanced key-value store...
oct 11 19:06:38 daniel-VirtualBox systemd[1]: Started Advanced key-value store.
oct 11 19:12:54 daniel-VirtualBox systemd[1]: Stopping Advanced key-value store...
oct 11 19:12:54 daniel-VirtualBox systemd[1]: redis-server.service: Deactivated successfu
oct 11 19:12:54 daniel-VirtualBox systemd[1]: Stopped Advanced key-value store.
oct 11 19:12:54 daniel-VirtualBox systemd[1]: redis-server.service: Consumed 1.240s CPU t
daniel@daniel-VirtualBox:~$

```
daniel@daniel-VirtualBox:~$ redis-cli -p 26379 SENTINEL get-master-addr-by-name redis-master
1) "192.168.56.11"
2) "6379"
daniel@daniel-VirtualBox:~$ sudo journalctl -u redis-sentinel -f
[sudo] contraseña para daniel:
oct 11 19:13:24 daniel-VirtualBox redis-sentinel[756]: 756:X 11 Oct 2025 19:13:24.422 # +sel
ected-slave slave 192.168.56.12:6379 192.168.56.12 6379 @ redis-master 192.168.56.11 6379
oct 11 19:13:24 daniel-VirtualBox redis-sentinel[756]: 756:X 11 Oct 2025 19:13:24.422 * +fai
lover-state-send-slaveof-noone slave 192.168.56.12:6379 192.168.56.12 6379 @ redis-master 19
2.168.56.11 6379
oct 11 19:13:24 daniel-VirtualBox redis-sentinel[756]: 756:X 11 Oct 2025 19:13:24.481 * +fai
lover-state-wait-promotion slave 192.168.56.12:6379 192.168.56.12 6379 @ redis-master 192.16
8.56.11 6379
oct 11 19:13:25 daniel-VirtualBox redis-sentinel[756]: 756:X 11 Oct 2025 19:13:25.338 # +pro
moted-slave slave 192.168.56.12:6379 192.168.56.12 6379 @ redis-master 192.168.56.11 6379
oct 11 19:13:25 daniel-VirtualBox redis-sentinel[756]: 756:X 11 Oct 2025 19:13:25.338 # +fai
lover-state-reconf-slaves master redis-master 192.168.56.11 6379
oct 11 19:13:25 daniel-VirtualBox redis-sentinel[756]: 756:X 11 Oct 2025 19:13:25.380 # +fai
lover-end master redis-master 192.168.56.11 6379
oct 11 19:13:25 daniel-VirtualBox redis-sentinel[756]: 756:X 11 Oct 2025 19:13:25.380 # +swi
tch-master redis-master 192.168.56.11 6379 192.168.56.12 6379
oct 11 19:13:25 daniel-VirtualBox redis-sentinel[756]: 756:X 11 Oct 2025 19:13:25.380 * +sla
ve slave 192.168.56.11:6379 192.168.56.11 6379 @ redis-master 192.168.56.12 6379
oct 11 19:13:26 daniel-VirtualBox redis-sentinel[756]: 756:X 11 Oct 2025 19:13:26.559 # +new
-epoch 2
oct 11 19:13:55 daniel-VirtualBox redis-sentinel[756]: 756:X 11 Oct 2025 19:13:55.430 # +sdo
wn slave 192.168.56.11:6379 192.168.56.11 6379 @ redis-master 192.168.56.12 6379
```

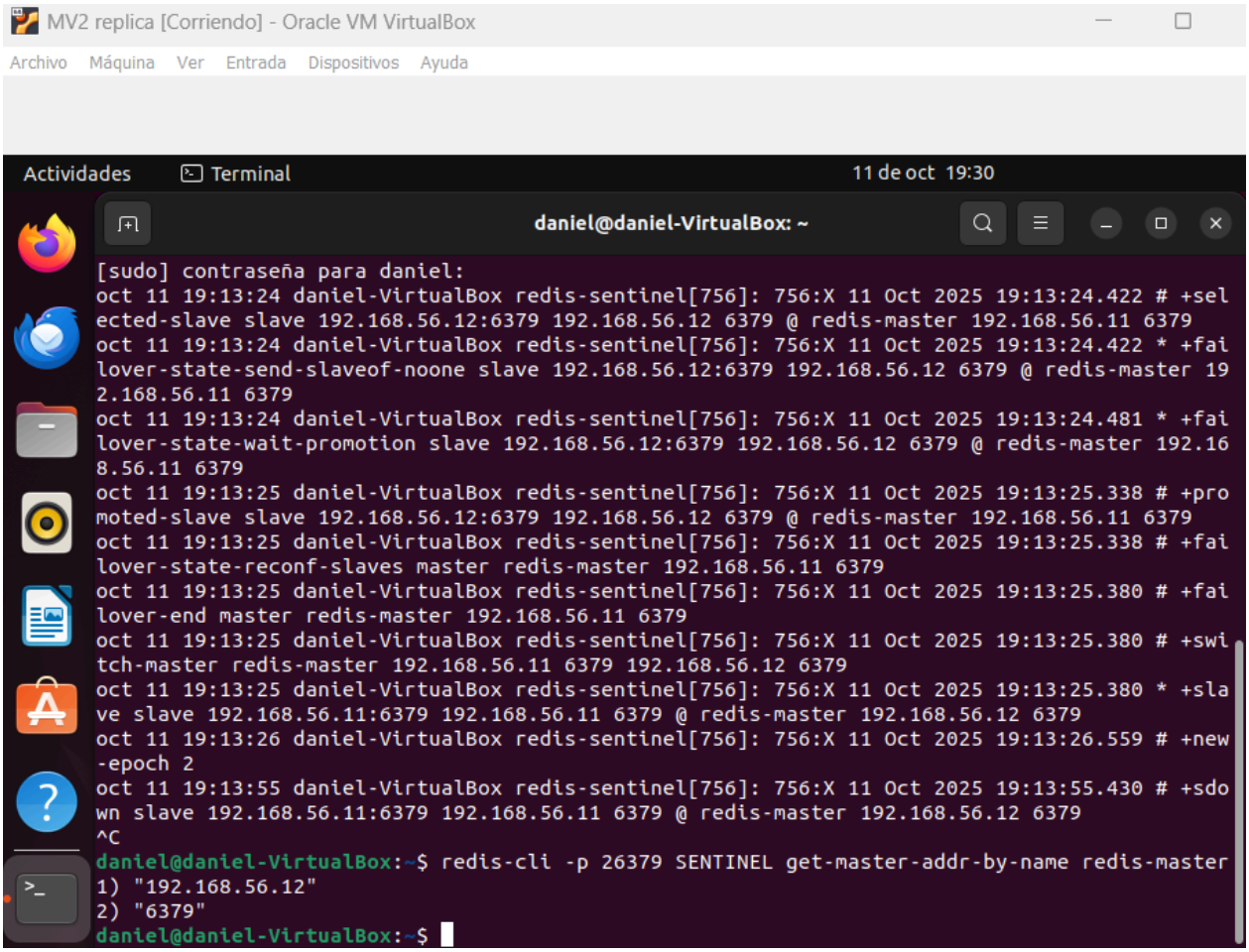
En cualquiera de las máquinas con Sentinel, mira los logs en vivo con los comandos:

sudo journalctl -u redis-sentinel -f

Esto muestra claramente cómo Sentinel detecta el fallo, elige una réplica, la promueve a master y actualiza la configuración de todos.

redis-cli -p 26379 SENTINEL get-master-addr-by-name redis-master

¡Significa que la réplica (192.168.56.12) fue promovida a master automáticamente!



4- Seguridad y cifrado

4.1 Autenticación de usuarios mediante requirepass

Comprobamos que la contraseña ya está modificada anteriormente y probamos la conexión:

```
daniel@daniel-VirtualBox:~$ redis-cli -a MiClaveSegura123 ping
Warning: Using a password with '-a' or '-u' option on the command line interface
 may not be safe.
PONG
```

Entramos al servidor para comprobar que pide contraseña:

CONFIG GET requirepass

```
daniel@daniel-VirtualBox:~$ redis-cli -h 192.168.56.11 -p 6379
192.168.56.11:6379> CONFIG GET requirepass
(error) NOAUTH Authentication required.
192.168.56.11:6379>
```

A continuación ponemos la contraseña y debería dejarnos entrar usando AUTH “contraseña”, mostrándonos el mensaje “Ok”:

```
daniel@daniel-VirtualBox:~$ redis-cli -h 192.168.56.11 -p 6379
192.168.56.11:6379> CONFIG GET requirepass
(error) NOAUTH Authentication required.
192.168.56.11:6379> AUTH MiClaveSegura123
OK
192.168.56.11:6379> CONFIG GET requirepass
1) "requirepass"
2) "MiClaveSegura123"
192.168.56.11:6379>
```

4.2 Cifrado TLS nativo

Comprobamos la versión de redis para ver si soporta TLS (debe ser la versión 6.0.9 o superior):

```
daniel@daniel-VirtualBox:~$ redis-server --version
Redis server v=6.0.16 sha=00000000:0 malloc=jemalloc-5.2.1 bits=64 build=a3fdef4
4459b3ad6
daniel@daniel-VirtualBox:~$
```

Vamos a generar un **CA (autoridad certificadora)** y tres pares de certificados (uno por nodo Redis).
Se puede hacer desde una sola máquina (por ejemplo la master 192.168.56.12):

4.2.1 Crear directorios para la CA

Vamos a crear el archivo de configuración openssl-san.conf que usaremos para generar el CSR del servidor con el campo **Subject Alternative Name (SAN)**, que es lo que Redis necesita para validar correctamente el certificado.

cd /redis-tls

nano openssl-san.conf

```
  GNU nano 6.2                    openssl-san.conf
[ req ]
default_bits        = 2048
prompt              = no
default_md          = sha256
req_extensions      = req_ext
distinguished_name = dn

[ dn ]
C  = ES
ST = Sevilla
L  = Sevilla
O  = IES Triana
OU = Educación
CN = 192.168.56.12

[ req_ext ]
subjectAltName = @alt_names

[ alt_names ]
IP.1 = 192.168.56.12
```

Este archivo define:

Los datos del certificado (CN, organización, etc.)

El campo SAN con la IP del servidor (192.168.56.12), que es lo que redis-cli necesita para validar correctamente.

Ahora debemos:

Crear la clave del servidor:

openssl genrsa -out server.key 2048

Generar el CSR usando el archivo:

openssl req -new -key server.key -out server.csr -config openssl-san.conf

Firmar el certificado del servidor con la CA

openssl x509 -req -in server.csr \

-CA ca.crt -CAkey ca.key -CAcreateserial \

-out server.crt -days 365 -sha256 \

-extfile openssl-san.conf -extensions req_ext

Verificar que el SAN está incluido openssl x509 -in server.crt -text -noout | grep -A1 "Subject Alternative Name"

```
root@daniel-VirtualBox:/redis-tls# openssl x509 -in server.crt -text -noout
| grep -A1 "Subject Alternative Name"
            X509v3 Subject Alternative Name:
                IP Address:192.168.56.12
root@daniel-VirtualBox:/redis-tls# nano openssl-san.conf
root@daniel-VirtualBox:/redis-tls#
```

Prueba la conexión con redis-cli

redis-cli --tls \

--cacert /etc/ssl/redis/ca.crt \

-h 192.168.56.12 -p 6379

```
root@daniel-VirtualBox:/redis-tls# redis-cli --tls \
> --cacert /etc/ssl/redis/ca.crt \
> -h 192.168.56.12 -p 6379
192.168.56.12:6379>
```

Prueba comandos básicos para confirmar que Redis responde:

```
root@daniel-VirtualBox:/redis-tls# redis-cli --tls --cacert /etc/ssl/redis/c
a.crt \
> -a MiClaveSegura123 \
> -h 192.168.56.12 -p 6379
Warning: Using a password with '-a' or '-u' option on the command line inter
face may not be safe.
192.168.56.12:6379> PING
PONG
192.168.56.12:6379> SET saludo "Hola daniel"
OK
192.168.56.12:6379> GET saludo
"Hola daniel"
192.168.56.12:6379>
```

4.3 Limitación de acceso por IP y firewall (iptables / UFW)

Usar iptables para permitir solo ciertas IPs

Supongamos que solo quieres permitir acceso desde 192.168.56.11 (máquina esclava) y bloquear todo lo demás:

```
root@daniel-VirtualBox:/home/daniel# iptables -A INPUT -p tcp --dport 6379 -
s 192.168.56.11 -j ACCEPT
root@daniel-VirtualBox:/home/daniel# iptables -A INPUT -p tcp --dport 6379 -
j DROP
root@daniel-VirtualBox:/home/daniel#
```

Desde una IP no permitida, debe fallar con Connection refused o Timeout.

Probamos desde la máquina 192.168.56.11:

```
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
    link/ether 08:00:27:50:8d:5d brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.11/24 brd 192.168.56.255 scope global noprefixroute enp0s8
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe50:8d5d/64 scope link
       valid_lft forever preferred_lft forever
daniel@daniel-VirtualBox:~$ sudo redis-cli --tls -h 192.168.56.12 -p 6379 -a MiC
laveSegura123
Warning: Using a password with '-a' or '-u' option on the command line interface
 may not be safe.
Could not connect to Redis at 192.168.56.12:6379: SSL_connect failed: certificat
e verify failed
not connected>
```

5- Creación de datos en la BBDD

5.1 Creación de datos en nodo Maestro:

Creamos datos sen la base de datos entrando en redis-cli con el comando:

redis-cli -h 192.168.56.11 -p 6379 -a MiClaveSegura123

A continuación creamos claves, listas y hashes:

```
daniel@daniel-VirtualBox:~$ redis-cli -h 192.168.56.11 -p 6379 -a MiClaveSegura123
Warning: Using a password with '-a' or '-u' option on the command line interface may not be sa
fe.
192.168.56.11:6379> SET usuario:1001 "Daniel"
OK
192.168.56.11:6379> SET usuario:1001 "Marina"
OK
192.168.56.11:6379> HSET producto:2001 nombre "laptop" precio "1200"
(integer) 2
192.168.56.11:6379> LPUSH cola:tareas "backup" "deploy" "monitor"
(integer) 3
```

Claves simples (Strings)

- Son el tipo más básico en Redis.
- Guardan un valor único asociado a una clave.

Hashes

- Son como un **diccionario** o **mapa** dentro de una clave.
- Permiten agrupar varios campos y valores bajo una misma clave.

Listas

- Son colecciones ordenadas de valores (tipo array).
- Permiten insertar elementos al inicio o al final.

5.2 Verificación de creación de datos

En el nodo maestro:

```
192.168.56.11:6379> GET usuario:1001
"Marina"
192.168.56.11:6379> HGETALL producto:2001
1) "nombre"
2) "laptop"
3) "precio"
4) "1200"
192.168.56.11:6379> LRANGE cola:tareas 0 -1
1) "monitor"
2) "deploy"
3) "backup"
```

En el nodo esclavo (192.168.56.12) y conectamos con tls debemos ver los datos que creamos anteriormente, por ejemplo el usuario:

```
daniel@daniel-VirtualBox:~$ redis-cli --tls --cert /etc/ssl/redis/server.crt --key /etc/ssl/redis
/server.key --cacert /etc/ssl/redis/ca.crt -h 192.168.56.12 -p 6379 -a MiClaveSegura123
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
192.168.56.12:6379> GET usuario:1001
"Marina"
192.168.56.12:6379>
```

6- Backups automáticos

6.1 Crear scripts de backup que ejecuten:

redis-cli SAVE

Preparación del entorno

Directorio de backups:

sudo mkdir -p /var/backups/redis

sudo chown redis:redis /var/backups/redis

sudo chmod 750 /var/backups/redis

Script de backup con BGSAVE y copia del RDB

Creamos el script y damos permisos de ejecución.

sudo nano /usr/local/bin/backup_redis.sh

```
#!/usr/bin/env bash

set -euo pipefail

# Configuración

REDIS_CLI="/usr/bin/redis-cli --tls \

  --cacert /etc/ssl/redis/ca.crt \

  --cert /etc/ssl/redis/redis-client.crt \
```

```
 --key /etc/ssl/redis/redis-client.key \
 -a MiClaveSegura123"

DATA_DIR="/var/lib/redis"
RDB_FILE="${DATA_DIR}/dump.rdb"
BACKUP_DIR="/var/backups/redis"
LOG_FILE="/var/log/redis_backup.log"
DATE="$(date +'%Y%m%d_%H%M%S')"
BACKUP_BASE="dump_${DATE}.rdb"
COMPRESS=true       # true/false
ENCRYPT=false       # true/false
GPG_RECIPIENT=""     # email/ID de clave pública si ENCRYPT=true (o usar -c simétrico)
RETENTION_DAYS=7     # borrar >7 días
MAX_FILES=30        # mantener como máximo 30 backups
TIMEOUT_SEC=120      # timeout para esperar BGSAVE

# Funciones auxiliares
log() { echo "[$(date +'%F %T')] $*" | tee -a "$LOG_FILE"; }

fail() { log "ERROR: $*"; exit 1; }

# Preparación
mkdir -p "$BACKUP_DIR"
touch "$LOG_FILE"
```

```
chmod 640 "$LOG_FILE"


# 1) Forzar snapshot y esperar
log "Iniciando BGSAVE…"
$REDIS_CLI BGSAVE || fail "BGSAVE falló al iniciar"


# Espera a que 'rdb_bgsave_in_progress' pase a 0 o timeout
start_ts=$(date +%s)
while true; do
  in_progress=$($REDIS_CLI INFO persistence | awk -F: '/rdb_bgsave_in_progress/ {print $2}' | tr -d '\r')
  if [[ "$in_progress" == "0" ]]; then
    log "BGSAVE completado"
    break
  fi
  if (( $(date +%s) - start_ts > TIMEOUT_SEC )); then
    fail "Timeout esperando BGSAVE ($TIMEOUT_SEC s)"
  fi
  sleep 2
done


# 2) Copiar RDB con timestamp
target="${BACKUP_DIR}/${BACKUP_BASE}"
cp "$RDB_FILE" "$target"
log "Copiado RDB a ${target}"
```

```
# 3) Verificación básica del RDB
if command -v redis-check-rdb >/dev/null 2>&1; then
  redis-check-rdb "$target" >/dev/null && log "Verificación OK: ${target}" || fail "Verificación RDB fallida: ${target}"
else
  log "Aviso: redis-check-rdb no encontrado, saltando verificación"
fi

# 4) Compresión
if [[ "$COMPRESS" == "true" ]]; then
  gzip -9 "$target"
  target="${target}.gz"
  log "Comprimido: ${target}"
fi

# 5) Cifrado (opcional)
if [[ "$ENCRYPT" == "true" ]]; then
  if [[ -n "$GPG_RECIPIENT" ]]; then
    gpg --batch --yes -o "${target}.gpg" --encrypt -r "$GPG_RECIPIENT" "$target"
  else
    # Simétrico (pide passphrase por STDIN si se integra con agente)
    gpg --batch --yes -o "${target}.gpg" -c "$target"
  fi
  rm -f "$target"
```

```
  target="${target}.gpg"
  log "Cifrado: ${target}"
fi

# 6) Retención por días
find "$BACKUP_DIR" -type f -mtime +"$RETENTION_DAYS" -print -delete | tee -a "$LOG_FILE"
log "Retención aplicada: >${RETENTION_DAYS} días"

# 7) Retención por número máximo
count=$(ls -1 "$BACKUP_DIR" | wc -l)
if (( count > MAX_FILES )); then
  purge=$((count - MAX_FILES))
  ls -1t "$BACKUP_DIR" | tail -n "$purge" | while read -r f; do
    rm -f "$BACKUP_DIR/$f"
    log "Purgado por límite: $f"
  done
fi

log "Backup finalizado: ${target}"
```

Ahora ejecutamos el script de backup con:

sudo /usr/local/bin/redis_backup.sh

```
root@daniel-VirtualBox:/etc/ssl/redis# /usr/local/bin/redis_backup.sh
[2025-11-28 09:26:52] Iniciando BGSAVE…
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
Background saving started
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
[2025-11-28 09:26:54] BGSAVE completado
[2025-11-28 09:26:54] Copiado RDB a /var/backups/redis/dump_20251128_092652.rdb
[2025-11-28 09:26:54] Verificación OK: /var/backups/redis/dump_20251128_092652.rdb
[2025-11-28 09:26:54] Comprimido: /var/backups/redis/dump_20251128_092652.rdb.gz
[2025-11-28 09:26:54] Retención aplicada: >7 días
[2025-11-28 09:26:54] Backup finalizado: /var/backups/redis/dump_20251128_092652.rdb.gz
root@daniel-VirtualBox:/etc/ssl/redis#
```

Al final obtendremos un archivo en /var/backups/redis/ con el timestamp en el nombre.

Puedes comprobarlo con:

ls -lh /var/backups/redis/

```
root@daniel-VirtualBox:/etc/ssl/redis# ls -lh /var/backups/redis/
total 4,0K
-rw-r----- 1 root root 256 nov 28 09:26 dump_20251128_092652.rdb.gz
root@daniel-VirtualBox:/etc/ssl/redis#
```

6.2 Programar backups con cron

Editamos el cron del usuario root:

sudo crontab -e

Añade una línea para que se ejecute cada noche a las 02:00:

0 2 * * * /usr/local/bin/redis_backup.sh

Guarda y confirma con:

sudo crontab -l

6.3 Verificación de restauración en nodo esclavo

Paso 1: Copiar el archivo de backup al esclavo

Desde MV1 (master), copia el archivo .rdb al MV2:

En mi caso:

scp /var/backups/redis/dump_20251128_093002.rdb daniel@192.168.56.12:/home/daniel/

```
root@daniel-VirtualBox:/var/backups/redis# scp /var/backups/redis/dump_20251128_092652.rdb.gz daniel@192.168.5
6.12:/home/daniel/
The authenticity of host '192.168.56.12 (192.168.56.12)' can't be established.
ED25519 key fingerprint is SHA256:lTK1j9wYEZlkYvk89g8XWELmFdUO1Y0lVgkKZpokXKU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.12' (ED25519) to the list of known hosts.
daniel@192.168.56.12's password:
dump_20251128_092652.rdb.gz                                   100%  256    43.5KB/s   00:00
```

Paso 2: Detener Redis en MV2

sudo systemctl stop redis

Paso 3: Reemplazar el archivo dump.rdb

sudo cp /home/daniel/dump_20251128_093002.rdb /var/lib/redis/dump.rdb

sudo chown redis:redis /var/lib/redis/dump.rdb

Paso 4: Arrancar Redis en MV2

sudo systemctl start redis

Paso 5: Verificar que cargó el snapshot

redis-cli --tls \

 --cacert /etc/ssl/redis/ca.crt \

 --cert /etc/ssl/redis/redis-client.crt \

 --key /etc/ssl/redis/redis-client.key -a MiClaveSegura123

Y ejecutamos:

info persistance (estadísticas relacionadas con la persistencia de datos en Redis)

```
root@daniel-VirtualBox:/var/backups/redis# redis-cli --tls   --cacert /etc/ssl/redis/ca.crt   --cert /etc/ssl/
redis/redis-client.crt   --key /etc/ssl/redis/redis-client.key -a MiClaveSegura123
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
127.0.0.1:6379> info persistence
# Persistence
loading:0
rdb_changes_since_last_save:0
rdb_bgsave_in_progress:0
rdb_last_save_time:1764318412
rdb_last_bgsave_status:ok
rdb_last_bgsave_time_sec:0
rdb_current_bgsave_time_sec:-1
rdb_last_cow_size:651264
aof_enabled:1
aof_rewrite_in_progress:0
aof_rewrite_scheduled:0
aof_last_rewrite_time_sec:-1
aof_current_rewrite_time_sec:-1
aof_last_bgrewrite_status:ok
aof_last_write_status:ok
aof_last_cow_size:0
module_fork_in_progress:0
module_fork_last_cow_size:0
aof_current_size:354
aof_base_size:354
aof_pending_rewrite:0
aof_buffer_length:0
aof_rewrite_buffer_length:0
aof_pending_bio_fsync:0
aof_delayed_fsync:0
```

*PRUEBAS

Crear datos y verificarlos en otra máquina

Máquina máster crea la clave “prueba” con el contenido “hola desde el máster” → redis-cli -h 192.168.56.11 -p 6379 -a MiClaveSegura123 set prueba "hola desde el master"

Comprobamos desde esclavo → redis-cli -h 192.168.56.11 -p 6379 -a MiClaveSegura123 get prueba

Failover

Configuración en mv3 sentinel:

port 26379

sentinel monitor redis-master 192.168.56.11 6379 2

sentinel auth-pass redis-master MiClaveSegura123

Arrancamos sentinel y se simula caída en el master (mv1).

Miramos los logs en la sentinel: journalctl -u redis-sentinel -f

Y comprobamos que el rol en la mv2 (esclavo) ahora es máster.

4.- CONCLUSIONES

Aprendizajes

Antes de realizar el proyecto de Redis había visto y usado otras bases de datos pero con Redis he conseguido entender más a fondo el funcionamiento de una supuesta base de datos con distintos nodos. Además de comprender el funcionamiento interno de las máquinas donde se alojan y de cómo manejar posibles fallos.

Dificultades

Para poder aprender el funcionamiento anteriormente mencionado he tenido que pasar por varias etapas de un poco de frustración por algunos fallos que no entendía. El failover ha sido dónde más dificultad he encontrado ya que había que configurar todas las máquinas con parámetros exactos y coordinados entre sí y para volver a restablecer la máquina maestro y volver a darle ese rol.

Posibles mejoras

Debido a las bajas pretensiones del PC donde hice el proyecto tuve que reducir las máquinas virtuales, esta estructura quedaría más completa y mejor repartida con un par de máquinas más, una de ellas por ejemplo encargándose únicamente de Sentinel, en mi caso tuve que usar Sentinel en todas para que no se sobrecargase ninguna y se cayeran.